Unidad 2

Variables y tipos de datos

Qué caracteriza a JavaScript, cómo se declaran variables con let y const y qué tipos primitivos existen.

El ADN de JavaScript

Antes de escribir código conviene saber con qué clase de lenguaje tratamos. JavaScript es:

Característica	Qué significa en la práctica
Ligero	Poca sintaxis obligatoria. Un programa puede ser una sola línea
Interpretado	No se compila antes: el navegador lo lee y lo ejecuta según llega
Dinámico	Las variables pueden cambiar de tipo durante la ejecución
Monohilo	Ejecuta una instrucción cada vez. Lo demás se organiza con eventos y asincronía
Multiparadigma	Admite programar con funciones, con objetos o mezclando ambos
Débilmente tipado	No se declara el tipo de las variables; el lenguaje lo deduce y convierte cuando hace falta
Case sensitive	`edad`, `Edad` y `EDAD` son tres variables distintas
Basado en prototipos	Los objetos heredan directamente de otros objetos, no de clases (aunque existe la sintaxis `class`)

En el lado del cliente, JavaScript se ejecuta en el navegador para responder a lo que hace el usuario: clics, teclas, envío de formularios, cualquier cambio en la interfaz.

Dónde va el código

Hay dos formas de meter JavaScript en una página:

```
<!-- Incrustado: dentro del propio HTML -->
<script>
  console.log('Hola desde el HTML');
</script>

<!-- Externo: en un archivo .js aparte -->
<script src="app.js"></script>
```

La opción externa es la preferida: separa el comportamiento del contenido, el mismo archivo se reutiliza en varias páginas, y el navegador puede guardarlo en caché. Lo incrustado sirve para pruebas rápidas.

La consola

Todos los navegadores traen una consola en las herramientas de desarrollo (F12 o `Cmd+Opt+I`). Desde el código se escribe en ella con los métodos de `console`:

```
console.log('Mensaje normal');
console.warn('Aviso: algo raro, pero sigue');
console.error('Error: algo ha fallado');
```

Es la herramienta principal para ver qué está haciendo el programa mientras se desarrolla. En este sitio, la salida de `console` aparece en el panel de cada ejercicio.

Variables

Una variable es un espacio de memoria con nombre donde se guarda un valor. Se declaran de tres maneras:

Palabra	Alcance	¿Se puede reasignar?	Cuándo usarla
`var`	Función completa	Sí	Nunca en código nuevo. Es la forma antigua, con reglas de alcance que dan sorpresas
`let`	Bloque `{ … }` donde se declara	Sí	Cuando el valor va a cambiar
`const`	Bloque `{ … }` donde se declara	No	Por defecto. Si no necesitas cambiarlo, es una constante

```
const nombre = 'Ana';   // no va a cambiar
let intentos = 0;       // va a cambiar
intentos = intentos + 1;

nombre = 'Eva';         // Error: no se puede reasignar una constante
```

Regla práctica: empieza siempre con `const`. Si el programa te obliga a reasignar, cámbialo a `let`. `var` no lo uses.

Los nombres se escriben en *camelCase* (`precioTotal`, `numeroDeAlumnos`), empiezan por letra y, como el lenguaje es *case sensitive*, `precioTotal` y `preciototal` son variables distintas.

Tipos de datos primitivos

Al ser débilmente tipado, JavaScript no pide declarar el tipo: lo deduce del valor. Los tipos primitivos son:

Tipo	Ejemplos	Notas
String	`'hola'`, `"hola"`	Texto. Entre comillas simples o dobles
Number	`42`, `3.14`, `-7`	Enteros y decimales comparten tipo. Incluye `Infinity` y `NaN` (*not a*

Tipo	Ejemplos	Notas
		number, resultado de una operación numérica sin sentido)
Boolean	`true` , `false`	Valores lógicos
Undefined	`undefined`	La variable existe pero aún no tiene valor
Null	`null`	Ausencia de valor, puesta a propósito
BigInt	`9007199254740993n`	Enteros más grandes de lo que Number maneja con precisión. La `n` final lo distingue

El operador `typeof` dice de qué tipo es un valor:

```
typeof 'hola';      // 'string'
typeof 42;          // 'number'
typeof true;        // 'boolean'
typeof undefined;   // 'undefined'
typeof 10n;         // 'bigint'
typeof null;        // 'object'  (un fallo histórico del lenguaje)
```

Conversión de tipos

Cuando una operación mezcla tipos, JavaScript convierte por su cuenta. A veces acierta y a veces no:

```
'5' + 3;      // '53'  → el + con un texto concatena
'5' * 3;      // 15    → el * sólo tiene sentido con números: convierte
'5' - 2;      // 3
'hola' * 2;   // NaN   → no hay forma de convertir 'hola'
```

Para no depender de esas reglas, se convierte a mano:

```
Number('12.5');     // 12.5
parseInt('12px');   // 12  (lee hasta el primer carácter no numérico)
```

```
String(42);         // '42'
Boolean(0);         // false
```

> ▶ ¿Qué valores se convierten a false?

Entrada de datos con prompt-sync

En el navegador existe `prompt()`, que abre un cuadro de diálogo. En Node.js, que es donde ejecutamos los programas en clase, no hay cuadros de diálogo ni `prompt`: se instala el paquete `prompt-sync` (`npm install prompt-sync`) y se importa al principio del programa.

```
// Importa el paquete prompt-sync
const prompt = require('prompt-sync')();

// Pide al usuario que ingrese su nombre
var nombre = prompt('¿Cuál es tu nombre? ');

// Saluda al usuario
console.log(`Hola, ${nombre}! Bienvenido al programa.`);
```

Dos cosas que hay que recordar: `prompt` devuelve **siempre un texto**, así que para operar con números hay que convertir con `Number()`; y la línea del `require` va antes de usarlo.

En los ejercicios de este sitio la línea del `require` funciona igual, pero no hay teclado: `prompt()` devuelve por orden unas entradas fijadas de antemano, que se indican en cada ejercicio, y los tests vuelven a ejecutar tu programa con otras entradas distintas. Lo que cuenta es que funcione con cualquier valor.

Ejemplo 1: declarar variables

Declara una constante nombre con el texto Ana, una variable edad con el número 17 y una variable matriculado con el valor lógico true. Después muestra las tres en la consola, cada una con su console.log, en ese orden.

```
// Declara aquí las tres variables

// Y muéstralas en la consola
```

Ctrl + Enter ejecuta con tests · Ctrl + Shift + Enter sólo el código · Tab indenta · Esc y luego Tab sale del editor

SALIDA DE CONSOLA

```
Sin salida todavía.
```

Los tests se muestran al ejecutar.

▶ Ver solución del ejemplo 1

Ejemplo 2: convertir texto a número

La variable precioTexto contiene un precio como texto. Crea una variable precio con el mismo valor convertido a número, y una variable total que sea precio multiplicado por 3. Muestra total en la consola.

```
const precioTexto = '12.5';

// Convierte precioTexto a número


// Calcula el total de tres unidades


console.log(total);
```

Ctrl + Enter ejecuta con tests · Ctrl + Shift + Enter sólo el código · Tab indenta · Esc y luego Tab sale del editor

SALIDA DE CONSOLA

```
Sin salida todavía.
```

Los tests se muestran al ejecutar.

▶ Ver solución del ejemplo 2

De este módulo

T2.1 Ejercicios: variables y tipos de datos
Doce programas con entrada de datos por prompt-sync, operaciones y salida por consola.
